

PaperHub — A Secure Document-Management and Review Platform

Project Report

Project title	PaperHub: A Role-Based Document Submission, Review, and Sharing Platform
Repository	<code>github.com/rajesh-1920/paperhub</code>
Module	<code>paperhub-saas</code> v1.0.0
Domain	Document Management Systems (DMS) / Lightweight Enterprise Content Management
Technology	Node.js, Express 5, vanilla JavaScript (ES), Tailwind CSS, MongoDB/GridFS, JSON file store
Document type	Software project report

Abstract

PaperHub is a web-based platform that lets organisations submit documents (PDFs), route them through a human review/approval workflow, and share or archive them under strict access control. It targets a Bangladeshi administrative context (admission forms, certificates, scholarship applications, office orders) but is domain-agnostic. The system implements production-grade authentication (bcrypt password hashing, JWT access tokens, and opaque server-revocable refresh tokens), role-based access control (RBAC) for three roles (user, officer, admin), per-resource ownership checks, an append-only audit log, and a per-viewer data-scoping layer. The backend is a small REST API over a pluggable storage façade that selects between a JSON file store (with on-disk binaries) and MongoDB with GridFS. Quality is enforced through a 153-case automated test suite (unit, integration against a live in-process server, and DOM-rendering tests) plus linting and formatting gates. This report documents the motivation, a literature review of the underlying techniques, the requirements, the architecture and design, the security model, the implementation, the testing strategy, and an evaluation of the result.

Table of Contents

1. Introduction
2. Literature Review
3. Requirements Analysis
4. System Architecture and Design
5. Security Design
6. Implementation
7. Testing and Quality Assurance

8. Results and Evaluation
9. Limitations and Future Work
10. Conclusion
11. References

1. Introduction

1.1 Background

Organisations routinely receive documents that must be **verified by a human, approved or rejected, tracked, and retained**. Spreadsheets and email threads scale poorly for this: they offer no access control, no audit trail, and no single source of truth. *Document Management Systems* (DMS) and the broader category of *Enterprise Content Management* (ECM) address this by combining storage, metadata, workflow, and security in one system [1].

PaperHub is a compact, self-hostable DMS built to demonstrate these capabilities end-to-end with **real security primitives** rather than mock authentication.

1.2 Problem Statement

Build a multi-user document platform that:

- authenticates users securely and keeps them signed in reliably;
- enforces *who can see and do what* (submitters, reviewers, administrators);
- moves a submitted document through a **review queue** to an approve/reject decision;
- stores the actual file bytes safely and serves them on demand;
- records an immutable activity trail; and
- never leaks one user’s data or credentials to another.

1.3 Objectives

#	Objective	Status
O1	Production-style authentication (hashed passwords, signed + revocable tokens)	Achieved
O2	Three-role RBAC with per-resource ownership enforcement	Achieved
O3	Document submission → review-queue → approve/reject workflow	Achieved
O4	Binary (PDF) storage with inline preview and download	Achieved
O5	Sharing, folders, tags, trash (soft delete), and version history	Achieved
O6	Append-only audit/activity log	Achieved
O7	Pluggable persistence (file store and MongoDB/GridFS)	Achieved
O8	Automated test coverage and CI-style quality gates	Achieved (153 tests)

1.4 Scope

In scope: a single-tenant web application with three roles, REST API, and two storage backends.
Out of scope: horizontal scaling, e-signature/PKI, OCR, and external identity federation (these are discussed in §9).

2. Literature Review

PaperHub synthesises several well-established bodies of work. The table below maps each technique to its primary reference and to how it informed the design; full citations with links are in §11.

Area	Key reference(s)	How it informed PaperHub
Document/ Enterprise Content Management	Päivärinta & Munkvold, <i>Enterprise Content Management</i> [1]	Justified the metadata + workflow + access-control triad as the core of a DMS.
Role-Based Access Control (RBAC)	Sandhu et al., <i>Role-Based Access Control Models</i> [2]; Ferraiolo et al., <i>Proposed NIST Standard for RBAC</i> [3]	Three-role model (user/officer/admin) with permission-to-role rather than permission-to-user assignment.
Token-based authentication	Jones et al., <i>JSON Web Token (RFC 7519)</i> [4]; Hardt, <i>OAuth 2.0 (RFC 6749)</i> [5]	Short-lived signed access tokens + long-lived opaque, server-revocable refresh tokens, mirroring the OAuth access/refresh split.
Password storage	Provos & Mazières, <i>A Future-Adaptable Password Scheme (bcrypt)</i> [6]; OWASP <i>Password Storage Cheat Sheet</i> [7]	bcrypt with a configurable cost factor; plaintext seeds are migrated to hashes on first login.
Identity & authentication guidance	NIST SP 800-63B, <i>Digital Identity Guidelines</i> [8]	Minimum password length, no forced rotation, verifier-side hashing.
Web application security	OWASP <i>Top 10 (2021)</i> [9]; OWASP ASVS [10]	Threat checklist (access control, auth, injection, misconfiguration) used to drive §5.
REST architectural style	Fielding, <i>Architectural Styles and the Design of Network-based Software Architectures</i> [11]; Pautasso et al., <i>RESTful Web Services vs. “Big” Web Services</i> [12]	Stateless, resource-oriented JSON API (<code>/api/files/:id/content</code> , <code>/api/dataset</code> , ...).
Software architecture	Bass, Clements & Kazman, <i>Software Architecture in Practice</i> [13]	Layered design and a storage façade isolating the persistence decision behind one interface.

2.1 Discussion

Access control. Sandhu et al. [2] formalised RBAC as assigning permissions to *roles* and users to roles, which simplifies administration versus per-user ACLs; the later NIST standard [3] consolidated this into a reference model. PaperHub follows the core RBAC model: officers and admins inherit broad permissions, while a regular user is constrained to their own slice. Where RBAC is too coarse (e.g., “share *this one* file with a colleague”), PaperHub augments it with a per-resource **ownership + ACL** check, a pattern consistent with combining RBAC and discretionary controls.

Authentication. Storing JWTs alone is convenient but cannot be revoked before expiry; storing only server-side sessions sacrifices statelessness. Following the OAuth 2.0 access/refresh separation [5], PaperHub issues a **stateless signed JWT** [4] for requests and a **stateful opaque refresh token** (only its SHA-256 hash is stored) that can be revoked on logout — combining the performance of stateless verification with the control of server-side revocation.

Password handling. Per Provos & Mazières [6] and OWASP/NIST guidance [7][8], passwords are never stored in plaintext; bcrypt’s adaptive cost resists hardware-accelerated cracking.

Security posture. The OWASP Top 10 [9] — led by *Broken Access Control* — directly shaped the data-scoping and write-policy layers described in §5.

3. Requirements Analysis

3.1 Functional Requirements

ID	Requirement	Role(s)
FR1	Register, log in, log out; remain signed in until explicit logout	All
FR2	Upload one or more PDF documents (drag-drop or browse)	User, Officer, Admin
FR3	View an uploaded document inline (modal preview) and download it	Owner / authorised
FR4	Submit a document into a review queue automatically on upload	User
FR5	Approve or reject a queued document; it then leaves the actionable queue	Officer, Admin
FR6	Organise files into folders and tags; move/rename/delete	Owner
FR7	Soft-delete to Trash; restore or permanently delete	Owner
FR8	Create and revoke shareable links (optionally password-protected)	Owner, Admin
FR9	Maintain version history of a document’s bytes	Owner
FR10	Per-role dashboards with live statistics	All
FR11	Admin: add, edit, and remove users	Admin
FR12	Append-only activity/audit log, viewable in-app	All (own) / Admin

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR1	Security	Hashed passwords; signed, revocable tokens; per-request authorization; no cross-user data leakage
NFR2	Reliability	A failed write must not silently corrupt state; uploads roll back cleanly on failure
NFR3	Portability	Run with zero external services (file store) or with MongoDB/GridFS, unchanged at the API layer
NFR4	Usability	Single-page-style navigation; clear empty/error states; responsive layout
NFR5	Maintainability	Automated tests, linting, and formatting enforced via one <code>check</code> command
NFR6	Performance	Stateless token verification; non-cacheable dataset reads to avoid stale counts

4. System Architecture and Design

4.1 High-Level Architecture



4.2 Technology Stack

Layer	Technology	Purpose
Frontend	Vanilla JavaScript (ES), HTML5, Tailwind CSS	No-framework SPA-style UI; shared global modules loaded in order
API	Node.js, Express 5	REST endpoints, middleware, static hosting
Auth	<code>jsonwebtoken</code> , <code>bcryptjs</code>	JWT signing/verification, password hashing
Storage (default)	JSON file + filesystem uploads/	Zero-dependency persistence; binaries as <code>.pdf</code> files
Storage (optional)	<code>mongodb</code> + GridFS	Database persistence; binaries of any size
Tooling	<code>node:test</code> , <code>jsdom</code> , ESLint, Prettier	Tests, DOM simulation, lint/format gates

4.3 Component Inventory

Module	File(s)	Responsibility
App/server	<code>server/index.js</code>	Wires middleware, routes, static hosting, binary endpoints
Storage façade	<code>server/db.js</code> , <code>server/stores/jsonStore.js</code> , <code>server/stores/mongoStore.js</code>	Single interface; selects backend by <code>MONGODB_URI</code>
Tokens	<code>server/auth/tokens.js</code>	JWT sign/verify; opaque refresh-token creation + hashing
Users/policy	<code>server/auth/users.js</code>	Sanitisation, per-viewer scoping, whole-dataset write policy
Ownership	<code>server/auth/ownership.js</code>	<code>canAccessResource</code> (owner / ACL / team)
Passwords	<code>server/auth/passwords.js</code>	<code>bcrypt</code> hashing, plaintext-to-hash migration
Audit	<code>server/auth/audit.js</code> , <code>server/routes/audit.js</code>	Append-only activity log
Quota / PDF	<code>server/quota.js</code> , <code>server/pdf.js</code>	Storage accounting; PDF page counting
Frontend data layer	<code>public/assets/js/utils.js</code>	Dataset fetch/persist, token handling, change bus
Frontend features	<code>file.js</code> , <code>review.js</code> , <code>share-view.js</code> , <code>activity.js</code> , <code>profile.js</code> , <code>main.js</code> , <code>navbar.js</code>	Pages and interactions

4.4 Data Model (selected entities)

Entity	Key fields	Notes
authAccount	id, email, passwordHash, role	Credentials; never leave the server
user	id, name, role, department, files[], reviews[], dashboard.stats	Public profile + embedded copies
file	id, name, ownerId, status, size, pageCount, hasContent, folderId, tags[], deletedAt	Metadata; bytes stored separately
reviewQueue[]	id, fileId, status, priority, reviewer, checklist[], comments[]	Submission tracking
folder / tag	id, ownerId, path / slug	Owner-scoped organisation
shareLink	token, resourceId, permission, passwordHash?, revoked	Token secret stored hashed
refreshToken	id, userId, tokenHash, expiresAt, revoked	Server-side, revocable
auditLog[]	id, ts, action, actorId, resourceId	Append-only

4.5 Key REST Endpoints

Method & path	Auth	Purpose
POST /api/auth/{login,register,refresh,logout}	mixed	Session lifecycle
GET /api/dataset	optional	Per-viewer-scoped application state
PUT /api/dataset	required	Persist mutated state (constrained by write policy)
PUT /api/files/:id/content	required	Store PDF bytes (≤ 2 GB; quota-checked)
GET /api/files/:id/content	—	Serve PDF bytes inline
POST /api/files/:id/versions	required	Archive + promote a new version
POST/GET/DELETE /api/share/*	mixed	Create / resolve / revoke share links
POST/GET /api/audit	required	Record / read activity

5. Security Design

Security is the project's centre of gravity. The controls below are mapped to OWASP Top 10 (2021) categories [9].

OWASP category	Control in PaperHub
A01 Broken Access Control	<code>requireAuth / authorize</code> middleware; <code>canAccessResource</code> ownership checks; a whole-dataset write policy that rebuilds non-owned data from the server copy so a non-admin PUT cannot alter others' records; per-viewer scoping of <code>GET /api/dataset</code> so a regular user only receives their own + shared resources.
A02 Cryptographic Failures	bcrypt password hashing [6]; share-link and refresh-token secrets stored only as SHA-256 hashes; credentials stripped from every client-facing projection.
A03 Injection	Parameterised storage access; strict <code>id</code> pattern (<code>^[A-Za-z0-9_-]+\$</code>) preventing path traversal on binary files; output escaping in the DOM layer.
A05 Security Misconfiguration	<code>x-powered-by</code> disabled; the runtime database file is blocked from static serving and is git-ignored; production boot refuses a weak <code>JWT_SECRET</code> .
A07 Identification & Auth Failures	Stateless JWT [4] + revocable refresh tokens; uniform 401s (no user enumeration); login-side bcrypt migration; long-lived sessions configurable per deployment.
A09 Logging & Monitoring	Append-only audit log of security-relevant actions (login, upload, share, remove).

5.1 Authentication Flow

1. **Login** verifies the bcrypt hash, then issues a signed JWT (`sub` , `role` , `name`) and an opaque refresh token `"<id>.<secret>"` ; only `sha256(secret)` is persisted.
2. **Requests** carry `Authorization: Bearer <jwt>` ; middleware verifies the signature statelessly.
3. **Refresh** exchanges a valid, non-revoked, non-expired refresh token for a new access token.
4. **Logout** revokes the refresh-token record server-side.

5.2 Authorization Layers

```
request → requireAuth (valid token?) → authorize(role) (RBAC)
    → canAccessResource (owner / ACL / team)
    → applyDatasetWritePolicy (non-admin: own slice only)
    → projectDatasetForViewer (read: own + shared only)
```


6. Implementation

6.1 Notable Features and Decisions

Feature	Implementation note
Pluggable storage	<code>server/db.js</code> selects <code>mongoStore</code> when <code>MONGODB_URI</code> is set, else <code>jsonStore</code> ; both expose an identical interface, so the API is storage-agnostic.
Binary handling	PDFs are stored outside the JSON document (filesystem or GridFS); metadata only references them, keeping the dataset small.
PDF page count	<code>server/pdf.js</code> derives the true page count from the bytes (<code>page-tree /Count</code> and <code>/Type /Page</code> , inflating <code>FlateDecode</code> object streams for PDF 1.5+).
Lenient PDF detection	<code>%PDF</code> is accepted anywhere in the first 1024 bytes (per the PDF spec), so files with a BOM/leading whitespace are not wrongly rejected.
Soft delete	<code>deletedAt</code> marks Trash; restore clears it; a global+embedded cross-check guarantees a deleted file never lingers on the files page.
Live data layer	A synchronous fetch/persist store with a <code>paperhub:change</code> event bus re-renders the open page after each mutation.
Seeded demo	The server seeds a populated demo dataset and bundles real, viewable PDFs into the content store on first boot.

6.2 Codebase Size

Area	Files	Lines (approx.)
Backend JS	<code>server/**/*.js</code>	~1,840
Frontend JS	<code>public/assets/js/**/*.js</code>	~7,300
CSS (excl. compiled Tailwind)	<code>public/assets/css/**</code>	~11,600
Tests	<code>tests/*.mjs</code> (34 files)	~3,680
HTML pages	<code>public/pages/**</code>	19 pages

7. Testing and Quality Assurance

Testing uses Node’s built-in `node:test` runner with `jsdom` for DOM-rendering tests, plus a live in-process Express server for integration tests. A single `npm run check` runs lint, format-check, and the full suite.

7.1 Test Categories

Category	Examples	What it verifies
Auth & config	login/refresh/logout, token TTLs, password migration	Session lifecycle and configuration defaults
Access control & privacy	write policy, per-viewer scoping, static-DB block	No cross-user reads/writes; no credential leakage
Files & upload	validation, page count, lenient PDF, partial-batch redirect	Correct, robust upload and rendering
Workflow	review-queue transitions, decided items leave the queue	Approve/reject correctness
Organisation	folders, tags, move/rename, trash restore/purge	Owner-scoped organisation and recovery
Storage	JSON store + GridFS round-trips, orphan pruning	Backend parity and integrity
Quota	limit resolution, unlimited-by-default	Storage accounting

7.2 Metrics

Metric	Value
Automated test cases	153 (all passing)
Test files	34
Quality gates	ESLint + Prettier + tests (one command)
Verification beyond unit tests	Real-server smoke runs (auth, upload, scoping, quota)

8. Results and Evaluation

PaperHub meets all eight objectives in [§1.3](#). Evaluation highlights:

- **Security.** Cross-user isolation was verified end-to-end: a regular user's scoped read returns only their own + shared files; a regular user's whole-dataset write cannot erase another user's data; the raw database file is not downloadable; credentials never appear in any client projection.
- **Correctness of workflow.** A decided (approved/rejected) document leaves the actionable review queue and cannot be re-decided; a deleted file leaves the list and lands in Trash with restore/purge working.
- **Robustness.** Uploads roll back cleanly on failure (no orphaned metadata or lingering review items), accept large and BOM-prefixed PDFs, and navigate to the files page whenever at least one file succeeds.

- **Portability.** The identical API and test suite pass against both the JSON file store and MongoDB/GridFS.

9. Limitations and Future Work

Limitation	Proposed future work
Single-tenant; one shared dataset document persisted as a whole	Move to granular, per-resource endpoints and optimistic concurrency
Long-lived JWTs trade revocability for convenience	Add token blacklisting / shorter access TTL with silent refresh in production
No e-signature, OCR, or full-text search	Integrate PDF text extraction + an index (e.g., Lucene-class engine)
No external identity federation	Add OAuth 2.0 / OIDC login [5]
Fully-compressed/encrypted PDFs may under-count pages	Use a dedicated PDF parsing library server-side

10. Conclusion

PaperHub demonstrates that a compact, framework-light codebase can deliver a **security-first** document-management workflow: real password hashing, signed-plus-revocable tokens, layered RBAC and ownership checks, per-viewer data scoping, an audit trail, and a storage-agnostic backend — all backed by a 153-case automated test suite. The design is grounded in established literature on RBAC [2][3], token-based authentication [4][5], adaptive password hashing [6], REST [11], and the OWASP security guidance [9][10]. The result is a functional, auditable platform and a reference implementation of the patterns it employs.

11. References

Links were valid at the time of writing. Where a DOI is given, it is the authoritative identifier.

- [1] T. Päivärinta and B. E. Munkvold, “Enterprise Content Management: An Integrated Perspective on Information Management,” *Proc. 38th Hawaii Int. Conf. on System Sciences (HICSS)*, IEEE, 2005. (Indexed in IEEE Xplore; search by title.)
- [2] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996. DOI: [10.1109/2.485845](https://doi.org/10.1109/2.485845). Author copy: [https://profsandhu.com/journals/computer/i94rbac\(org\).pdf](https://profsandhu.com/journals/computer/i94rbac(org).pdf)

- [3] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Trans. on Information and System Security (TISSEC)*, vol. 4, no. 3, pp. 224–274, 2001. DOI: [10.1145/501978.501980](https://doi.org/10.1145/501978.501980).
- [4] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [5] D. Hardt (Ed.), "The OAuth 2.0 Authorization Framework," IETF RFC 6749, 2012. <https://datatracker.ietf.org/doc/html/rfc6749>
- [6] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," *Proc. USENIX Annual Technical Conf.*, 1999. <https://www.usenix.org/legacy/events/usenix99/provos.html>
- [7] OWASP, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [8] P. A. Grassi et al., "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, 2017. <https://pages.nist.gov/800-63-3/sp800-63b.html>
- [9] OWASP, "OWASP Top 10 — 2021." <https://owasp.org/Top10/>
- [10] OWASP, "Application Security Verification Standard (ASVS)." <https://owasp.org/www-project-application-security-verification-standard/>
- [11] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [12] C. Pautasso, O. Zimmermann, and F. Leymann, "RESTful Web Services vs. 'Big' Web Services: Making the Right Architectural Decision," *Proc. 17th Int. Conf. on World Wide Web (WWW)*, 2008, pp. 805–814. DOI: [10.1145/1367497.1367606](https://doi.org/10.1145/1367497.1367606).
- [13] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Addison-Wesley, 2012. ISBN: 978-0321815736.

Report generated for the PaperHub project. Source code, test suite, and this document are maintained in the project repository.